Universidad ICESI

45 años

Contract-Driven Architecture & Enterprise Solutioning

From Architectural Plans to Code Contracts, Data Schemas, and Enterprise Test Gates

Session Agenda & Laboratory Objectives

- **Derive Formal Module Contracts:** Define explicit Preconditions (P), Postconditions (Q), and Class/Module Invariants (I) for key system components.
- **Design Internal Interfaces & Schemas:** Structure domain services using Hexagonal Architecture (Ports & Adapters) and concrete DTO/PostgreSQL schemas.
- **Scaffold Enterprise Test Workflows:** Integrate the BMad TEA framework (ATDD red-phase scaffolds, CI/CD quality gates, and NFR evidence audits).
- **Implement the Solutioning Stop-Point:** Complete solutioning BEFORE technical story creation to guarantee zero-ambiguity criteria.
- **Execute Implementation Readiness Audit:** Run bmad-sprint-planning compiler check to certify IR-PASS before releasing work to autonomous implementation agents.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

The Pedagogical Bridge: From Spec Review to Internal Solutioning

Unit 3: Spec Review & Macro Architecture (RAA3)

- Focus: High-level specifications, external APIs, and system context.
- Role: Auditor / Multi-Agent Council Lead.
- Tools: /bmad-prd validate, /bmad-review, /bmad-party-mode.
- Artifacts: PRD, High-level ARCHITECTURE-SPINE.md, Domain Aggregates.
- Key Outcome: Alignment on WHAT to build and macro system invariants (AD-n).

Unit 4: Contracts, Internal Design & Schemas (RAA4)

- Focus: Internal interfaces, behavioral contracts (P, Q, I), & DTO/SQL schemas.
- Role: System Solutioning Architect & Quality Engineer.
- Tools: /bmad-architecture, /bmad-testarch-*, /bmad-create-epics-and-stories.
- Artifacts: Hexagonal Ports & Adapters, Contracts, ATDD Scaffolds, Tech Stories.
- Key Outcome: Inviolable machine contracts HOW to implement before writing code.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

The Core Problem: Why AI Coding Fails Without Internal Contracts

1. The State Corruption Dilemma

LLM dev agents given high-level stories without precondition guards (P) write code that mutates invalid states (e.g. processing payment on cancelled orders).

2. Cost Escalation (1x → 100x)

Fixing contract gaps in Solutioning costs 1x. Discovering schema drift during coding costs 10x. Production failures cost 100x.

3. Schema & Interface Drift

Without explicit DTO schemas and Ports & Adapters interfaces, frontend and backend agents build incompatible payload structures.

Rule: High-level requirements describe business intent. Internal contracts (Preconditions, Postconditions, Invariants) turn intent into deterministic machine code constraints.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Meet Your Enterprise Solutioning Council: 4 Specialized Personas

Winston — System Architect

Handle: bmad-agent-architect

Owns Ports & Adapters placement, internal interfaces, DTO definitions, SQL schemas, and AD-n invariants.

Amelia — Senior Developer

Handle: bmad-agent-dev

Audits method signatures, clean code conventions, domain model feasibility, and module boundary cohesion.

Murat — Master Test Architect

Handle: bmad-tea

Owns ATDD red-phase scaffolds, CI quality pipelines, NFR evidence audits, and traceability matrices.

John — Product Manager

Handle: bmad-agent-pm

Ensures technical user stories trace directly back to verified contracts without scope creep or vanity features.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

The Enterprise Solutioning Pipeline (The Solutioning Stop-Point)

1. High-Level Spec

PRD & ARCH-SPINE (Unit 3)

2. Formal Contracts

Pre/Post/Invariants (P, Q, I)

3. Internal Interfaces

Ports & Adapters & Schemas

4. Enterprise Test Gates

ATDD & CI Scaffolds (TEA)

5. Technical Stories

bmad-create-epics-and-stories

6. STOP & Audit Gate

bmad-sprint-planning PASS

Solutioning halts strictly at Story Creation (bmad-create-epics-and-stories) and Sprint Readiness Audit (bmad-sprint-planning). Autonomous implementation agents (bmad-build) are NOT invoked during solutioning.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Today's 60-Minute Laboratory: 4-Phase Studio Workflow

Phase 1 (15 min)

Formal Contract Derivation

Command:
/bmad-architecture
/bmad-testarch-atdd

Define preconditions (P), postconditions (Q), & invariants (I) for core modules.

Phase 2 (15 min)

Internal Interface & Schemas

Command:
/bmad-architecture

Structure Ports & Adapters, DTO interfaces, and PostgreSQL migrations DDL.

Phase 3 (15 min)

Enterprise Test Scaffolding

Command:
/bmad-testarch-* (TEA Framework)

Scaffold ATDD red-phase test suites, CI quality gates, and NFR matrices.

Phase 4 (15 min)

Story Creation & Audit Gate

Command:
/bmad-create-epics-and-stories
/bmad-sprint-planning

Generate user stories & certify IR-PASS readiness.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Phase 1: Formal Behavioral Contracts (Preconditions, Postconditions, Invariants)

Preconditions (P) — Client Obligations

Caller MUST satisfy before execution. Guard clauses return HTTP 400/403 immediately on violation.

Example: assertCoHostOwnsEvse(guestId, evseId) -> evses.remote_site_id owner must equal guestId.

Postconditions (Q) — Supplier Guarantees

Service GUARANTEES upon completion. Event emissions, database state transitions, and audit records.

Example: handleTelegramCallback -> Token marked consumed AND dual Telegram chat messages edited.

Invariants (I) — Unalterable State Rules

Constraints that MUST remain true before and after every public operation.

Example: Host Priority Override -> Exactly ONE execution permitted per (event_type, event_id) pair.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Phase 2: Internal Interface Design & Architectural Placement

Driving Adapters (Controllers): Express routes (routes/bookings.ts) & Telegram Webhook Controller (webhooks/telegram.ts).

Inbound Ports (Use Cases): IAuthorizeArrivalUseCase, IHandleTelegramCallbackUseCase, IApproveBookingUseCase.

Core Domain & Invariants: Pure business logic: assertCoHostOwnsEvse predicate, TelegramConcurrencyArbitrator.

Outbound Ports (Interfaces): IActionTokenRepository, IBreakerCommandPublisher, ITelegramMessageEditor.

Driven Adapters (Infrastructure): PostgresActionTokenRepo, MqttBreakerPublisher (ssf/{site}/command/breaker).

Phase 2 Detail: Concrete Data Schemas & DTO Specifications

PostgreSQL Migration DDL (076_telegram_action_tokens.sql)

```
CREATE TABLE telegram_action_tokens (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    token TEXT UNIQUE NOT NULL,
    event_type TEXT NOT NULL,
    event_id UUID NOT NULL,
    host_chat_id TEXT, host_message_id TEXT,
    cohost_chat_id TEXT, cohost_message_id TEXT,
    expires_at TIMESTAMPTZ NOT NULL,
    consumed_at TIMESTAMPTZ,
    consumed_by TEXT
);
CREATE UNIQUE INDEX idx_tokens_active
ON telegram_action_tokens(token)
WHERE consumed_at IS NULL;
```

TypeScript DTO Specifications (co-host.dto.ts)

```
export interface AuthorizeArrivalDTO {
  siteId: string;
  bookingId: string;
  authorizeEntry: boolean;
  powerOnCharger: boolean;
  callerGuestProfileId?: string;
  callerUserId?: string;
}

export interface TelegramCallbackDTO {
  callbackQueryId: string;
  chatId: string;
  messageId: number;
  token: string;
  eventType: 'arrival_authorize' | 'booking_approve';
  eventId: string;
}
```

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Phase 3: Enterprise Test Architecture (The BMad TEA Framework)

Risk-Based Test Strategy (bmad-testarch-test-design [TD])

Maps system risks and defines coverage boundaries for critical workflows.

Test Framework Scaffolding (bmad-testarch-framework [TF])

Initializes production test harnesses (Playwright / Cypress / Vitest) with standard fixtures.

CI/CD Pipeline Quality Gates (bmad-testarch-ci [CI])

Configures automated quality pipelines to execute contract suites on every PR.

Acceptance TDD Scaffolds (bmad-testarch-atdd [AT])

Transforms formal contracts (P, Q, I) into failing red-phase acceptance test suites.

NFR Evidence Audit (bmad-testarch-nfr [NR])

Audits performance, security isolation, and throughput telemetry against baseline criteria.

Traceability & Gate Decision (bmad-testarch-trace [TR])

Generates Requirements-to-Test Traceability Matrix and certifies gate pass/fail.

Phase 3*: bmad-teach-me-testing

Session	Name	Est. Duration	Core Topics Covered
Session 1	Quick Start	30 min	TEA Lite overview, automated test workflows, running your first test scan.
Session 2	Core Concepts	45 min	Risk-based testing philosophy, priority tiers (P0–P3), Definition of Done (DoD), quality strategy.
Session 3	Architecture	60 min	Test framework architecture, Playwright/Cypress fixtures, network/API mocking patterns, clean test code.
Session 4	Test Design	60 min	Risk-based test planning workflow (bmad-testarch-test-design), testability reviews for architecture and epics.
Session 5	ATDD & Automation	60 min	Acceptance Test-Driven Design (bmad-testarch-atdd), red-phase test scaffolds, test automation expansion (bmad-testarch-automate).
Session 6	Quality & Traceability	45 min	Test quality auditing (bmad-testarch-test-review), Requirements-to-Test Traceability Matrix (bmad-testarch-trace), gate certification.
Session 7	Advanced Patterns	Ongoing	Self-directed deep dive into 59 specialized knowledge fragments (performance testing, security isolation, CI quality gates).

Phase 3 Detail: ATDD & Red-Phase Acceptance Test Scaffolding

Executable Red-Phase ATDD Suite (tests/atdd/co-host-contracts.test.ts)

```
describe('Unit 4 ATDD: Co-Host Operational Contracts', () => {
  describe('assertCoHostOwnsEvse Contract (Precondition P)', () => {
    it('RED: should allow execution when guest matches remote_sites.owner_guest_id', async () => {
      // Contract assertion: Ownership chain resolves successfully
    });

    it('RED: should throw FORBIDDEN_NOT_EVSE_OWNER (403) when guest does not own EVSE', async () => {
      // Contract assertion: Non-owner request rejected immediately at handler boundary
    });
  });

  describe('handleTelegramCallbackQuery Contract (Host Priority Override Invariant I)', () => {
    it('RED: should execute action and edit BOTH chat threads on first tap (Winner)', async () => {
      // Contract assertion: Token marked consumed, dual editMessageText dispatched
    });

    it('RED: should reject second tap with "Already resolved" and zero duplicate MQTT commands', async () => {
      // Contract assertion: Atomic UPDATE fails, loser receives idempotent toast
    });
  });
});
```

Phase 4: Technical Story Creation & The Readiness Gate

Contract-Bound User Stories

• Command: /bmad-create-epics-and-stories

• Derivation: Stories are generated directly from finalized contracts and schemas.

• Acceptance Criteria Structure:

- Given: Precondition P is satisfied & ATDD scaffold exists.

- When: API endpoint / Telegram callback is invoked with target DTO.

- Then: Postcondition Q holds & DB table / MQTT topic updated.

• Zero Ambiguity: Every story references exact DTO files, SQL migrations, and ATDD test paths.

Sprint Readiness Gate Audit (bmad-sprint-planning)

IR-FAIL (Blocking Stop): Unhandled contract gap or missing schema locks pipeline. Coding agents (/bmad-build) are hard-locked out.

IR-CONCERNS (Review Required): Non-critical NFR warning requires explicit architect sign-off in memlog.

IR-PASS (Autonomous Green Light): 100% contract-to-story traceability certified. Sprint tracking file generated. Implementation begins.

Defect Case 1 — Unprotected State Mutation & Missing Invariant Guard

Defective Implementation & Disaster

• Flaw: Handler processPayment(bookingId) mutates payment status without verifying booking state.

• Trigger: Client sends payment payload for a booking already marked CANCELLED or REFUNDED.

• Consequence: System charges credit card, emits payment success event, but booking remains cancelled. Financial audit discrepancy.

How Unit 4 Contracts & BMad Refactor It

• Formal Precondition P: Assert booking.status === 'PENDING_PAYMENT' before payment execution.

• Formal Invariant I: Sum(paidAmounts) <= booking.totalAmount.

• ATDD Scaffold: Scaffolds red-phase test processPayment_cancelled_booking_throws_400().

• Result: Dev agent (bmad-build) is constrained by contract guard; bug eliminated before coding.

Defect Case 2 — Concurrency Race Condition in Dual Telegram Callback

Defective Implementation & Disaster

• Flaw: Separate tokens issued for Host and Co-Host without atomic shared state.

• Trigger: Host and Co-Host tap [Authorize Entry] simultaneously on their phones.

• Consequence: Both callbacks succeed independently. Duplicate MQTT breaker ON commands published, corrupting arrival audit log.

How Unit 4 Contracts & BMad Refactor It

• Architectural Invariant AD-18: Atomic Single-Winner Token Update.

• Contract Mechanism: Single shared token string embedded in both inline keyboards.

• DB Update Predicate: UPDATE telegram_action_tokens SET consumed_at=NOW() WHERE token=$1 AND consumed_at IS NULL RETURNING *.

• Result: Database serializes race. First write wins; second tap receives idempotent 'Already resolved' toast.

Defect Case 3 — Data Schema Drift & Loose Payload Contracts

Defective Implementation & Disaster

• Flaw: Frontend sends payload { arrivalAuthorized: true } while backend expects { authorize_entry: true, power_on_charger: true }.

• Trigger: Frontend and backend agents implemented stories from vague text descriptions without shared DTO contract.

• Consequence: Driver arrival authorized on web UI, but smart breaker fails to energize. Driver stranded at gate.

How Unit 4 Contracts & BMad Refactor It

• Phase 2 DTO Binding: Formulate explicit TypeScript AuthorizeArrivalDTO interface before story creation.

• Schema Validation: API Gateway enforces JSON schema validation middleware on route entry.

• Story Linkage: Technical story AC explicitly mandates AuthorizeArrivalDTO contract compliance.

• Result: Zero payload mismatch during frontend/backend integration.

Defect Case 4 — Unverified Non-Functional Requirement (NFR Drift)

Defective Implementation & Disaster

• Flaw: Telegram webhook handler synchronously awaits MQTT broker confirmation before responding to Telegram API.

• Trigger: Network latency on remote breaker connection delays response beyond 30 seconds.

• Consequence: Telegram API times out, displays loading spinner error to user, and resends duplicate webhook queries.

How Unit 4 Contracts & BMad Refactor It

• NFR Audit (/bmad-testarch-nfr): Evaluates NFR-2 (answerCallbackQuery < 500ms).

• Contract Decoupling: Webhook handler invokes answerCallbackQuery immediately after atomic token update.

• Async Queueing: Breaker MQTT dispatch enqueued asynchronously in background task.

• Result: Zero Telegram timeout error spinners; 100% NFR compliance.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Classroom Facilitation Timeline & Milestones (80 Min Total)

00:00 - 00:15 (15 min) — Phase 1: Contract Derivation: Define P, Q, I clauses for core domain services using /bmad-architecture.

00:15 - 00:35 (20 min) — Phase 2: Ports & Schemas: Structure Hexagonal Ports & Adapters and write PostgreSQL migration DDL.

00:35 - 00:55 (20 min) — Phase 3: Test Scaffolding: Scaffold red-phase ATDD suites and CI quality gates using /bmad-testarch-*.

00:55 - 01:15 (20 min) — Phase 4: Story Creation & Gate: Run /bmad-create-epics-and-stories and certify IR-PASS readiness.

01:15 - 01:20 (5 min) — Session Wrap-up: Submit Worksheet 5 deliverables & review rubric evaluation criteria.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Teacher Interventions & Student Coaching Playbook

Phase 1 Trap: Vague Logic in Specs

Student Trap: Students write generic prose instead of formal P, Q, I contracts.

Teacher Coaching: Force them to write explicit Preconditions (P), Postconditions (Q), and Invariants (I) using elastic-free language.

Phase 2 Trap: Framework Coupling

Student Trap: Students couple domain logic directly to Express or PostgreSQL ORMs.

Teacher Coaching: Enforce Hexagonal Ports & Adapters separation: Core domain logic must never import Express or TypeORM.

Phase 3 Trap: Premature Coding

Student Trap: Students skip test scaffolding and attempt to start implementation.

Teacher Coaching: Remind students: Red-phase ATDD scaffolds must be generated BEFORE code. Tests define the contract boundary.

Phase 4 Trap: Bypassing Gate Failures

Student Trap: Students ignore gate warnings and claim 'it's close enough.'

Teacher Coaching: A gate FAIL is a compiler error. Challenge students to fix missing DTO links until bmad-sprint-planning prints IR-PASS.

UNIT 4 — CONTRACTS, INTERNAL DESIGN & ENTERPRISE SOLUTIONING

Summary: Solutioning as an Inviolable Machine Contract

1. Contracts Before Stories

Never generate user stories until Preconditions (P), Postconditions (Q), Invariants (I), and DTO schemas are mathematically defined.

2. Enterprise Test Gates

Scaffolding ATDD red-phase test suites and CI quality pipelines before coding guarantees zero-defect software delivery.

3. The Solutioning Stop-Point

Solutioning stops strictly at story creation with a certified IR-PASS. Never release unverified work to implementation agents.